

CHAPTER

Loops in Python — The for Loop

Learning Objectives

After studying this chapter, you will be able to:

- Understand what a loop is and why programmers use them.
- Write a basic for loop using correct Python syntax.
- Iterate over different sequences like strings, lists, tuples, sets and dictionaries.
- Use the range() function to repeat an action a fixed number of times.
- Build nested for loops to solve multi-level problems.
- Control the flow of a loop using the break, continue and pass statements.
- Combine a for loop with an else block for advanced use cases.

1. Introduction to the for Loop

Imagine you are a teacher and you need to greet every student in your classroom one by one. If there are 40 students, writing 40 separate print statements in your program would be boring, slow and full of repetition. Programmers face this kind of problem very often: doing the same work again and again for many items.

This is exactly where loops come in. A loop is a programming construct that tells the computer to repeat a block of instructions automatically. Python gives us two main loops: the for loop and the while loop. In this chapter, we will focus completely on the for loop.

What Does a for Loop Do?

A for loop in Python is used to go through (iterate over) each item in a collection — such as a list of numbers, the letters of a word, or a set of names — and run the same instructions for each of those items. You do not need to know in advance how many items are present; Python handles that automatically.

Key Idea

A for loop is best used when you already have a group of items (a sequence) and you want to do something with each item one at a time.

A Real-Life Analogy

Think of a cashier scanning items at a supermarket counter. Each product is placed on the belt, scanned, and the price is added to the total. The cashier does not know how many items are there; he simply keeps scanning until the belt is empty. A for loop works in the very same way — it keeps processing items one by one until there are no more items left.

Why Do We Need for Loops?

- They reduce the amount of code you have to write.
- They make programs shorter, cleaner and easier to read.
- They make it easy to handle collections of any size — ten items or ten thousand items, the code stays the same.
- They help avoid human mistakes caused by copying the same instructions many times.

Without a Loop (Repetitive Code)

```
print("Hello, Student 1")  
print("Hello, Student 2")
```

```
print("Hello, Student 3")  
print("Hello, Student 4")  
print("Hello, Student 5")
```

With a for Loop (Clean Code)

```
for number in range(1, 6):  
    print("Hello, Student", number)
```

Output:

```
Hello, Student 1  
Hello, Student 2  
Hello, Student 3  
Hello, Student 4  
Hello, Student 5
```

Both programs produce exactly the same output, but the second version uses only two lines of code. This is the real power of a for loop.



2. Syntax of the for Loop

Before we start writing many examples, let us first understand the basic structure (also called syntax) of a for loop in Python. Syntax simply means the set of rules you must follow while writing code, just like grammar rules in English.

The General Form

```
for variable in sequence:  
    # block of statements to be executed  
    statement_1  
    statement_2
```

Understanding Each Part

Part	Meaning	Example
for	A reserved keyword in Python that starts the loop.	for
variable	A name you choose. It temporarily holds the current item from the sequence.	item, num, name
in	Another keyword that connects the variable to the sequence.	in
sequence	The collection of items you want to go through.	list, string, range()
colon (:)	Marks the end of the loop header. It is compulsory.	:
indented block	The statements to repeat. They must be indented (usually 4 spaces).	print(item)

Important

In Python, indentation is not just for looks — it is a strict rule. All statements that belong to the loop must start at the same level of indentation. If you forget to indent, Python will raise an `IndentationError`.

A Simple First Example

```
fruits = ["apple", "banana", "mango"]  
  
for fruit in fruits:
```

```
print(fruit)
```

Output:

```
apple  
banana  
mango
```

Let us break down how Python executes this code:

1. Python looks at the list `fruits`, which has three items.
2. It picks the first item "apple" and stores it inside the variable `fruit`.
3. It then runs the indented statement — `print(fruit)` — which prints apple.
4. It moves to the next item "banana", stores it in `fruit`, and prints again.
5. It continues until every item has been used. Then the loop stops automatically.

How the for Loop Works — Step by Step

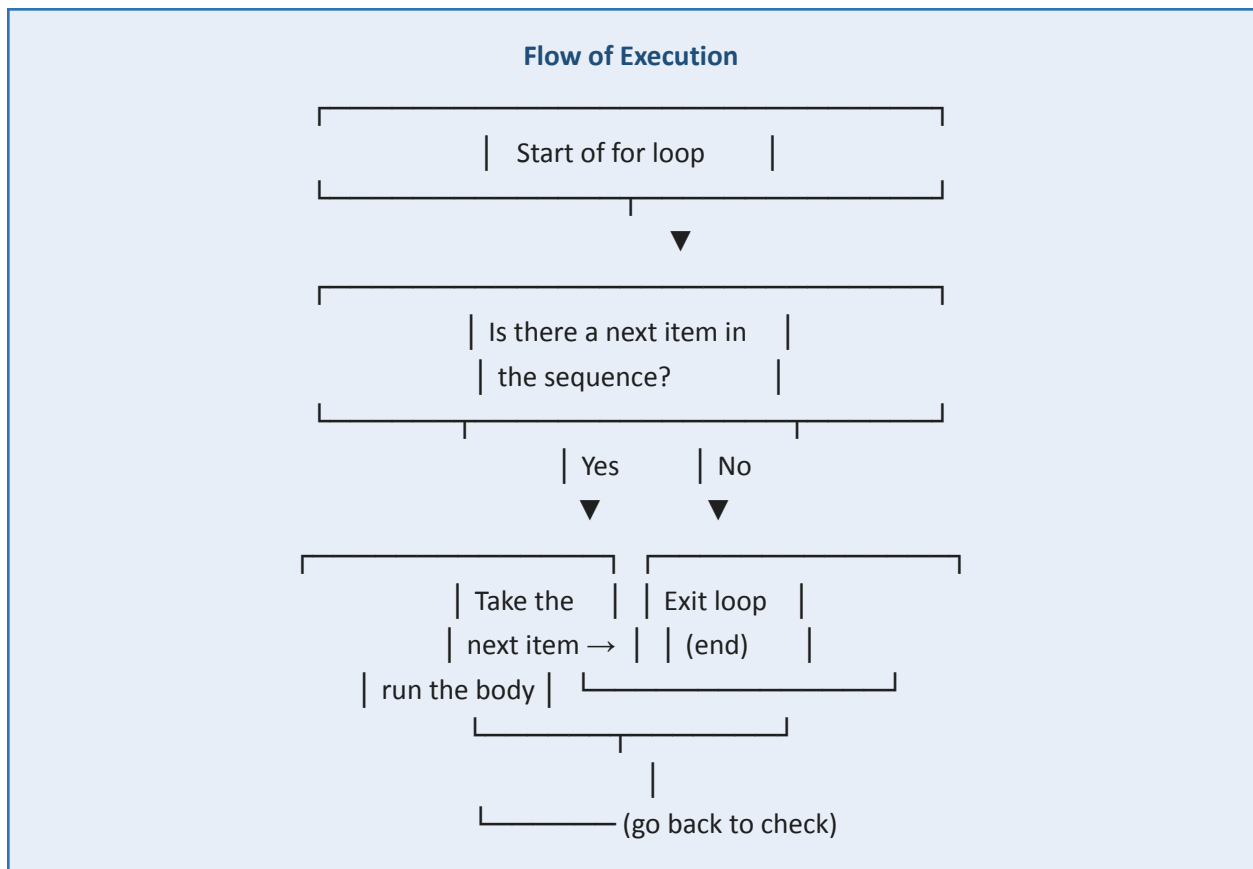


Fig. 2.1 — The working of a for loop.

3. Iterating Over a Sequence

The word iterate simply means "to go through items one by one". A sequence in Python is any collection of items that are kept in a particular order. Strings, lists, tuples and ranges are all sequences. The for loop is designed especially to work with them.

What Is Iteration?

Every time the loop picks one item from the sequence and executes the block of statements, one iteration is said to be complete. If the sequence has 5 items, the loop will run through 5 iterations.

Example: Iterating Over a List of Numbers

```
numbers = [10, 20, 30, 40, 50]

for num in numbers:
    print("Current number is:", num)
```

Output:

```
Current number is: 10
Current number is: 20
Current number is: 30
Current number is: 40
Current number is: 50
```

Here the list numbers has 5 elements, so the loop body runs 5 times. In each iteration, the variable num holds one element from the list.

Example: Performing a Calculation

Let us now add all the numbers in the list using a for loop.

```
numbers = [10, 20, 30, 40, 50]
total = 0

for num in numbers:
    total = total + num

print("The sum of all numbers is:", total)
```

Output:

```
The sum of all numbers is: 150
```

Notice how a variable called `total` is created before the loop starts. Inside the loop, every number is added to `total`. After the loop finishes, we print the final result. This is a very common pattern you will use many times in programming.

 Tip

Always create any helper variables (such as `total`, `count`, or `max_value`) before the loop begins. If you create them inside the loop, they will be reset every time and you will lose the earlier values.



4. Using range() with the for Loop

Sometimes you do not have a ready-made list of items. You simply want to repeat an action a fixed number of times — for example, print "Hello" 10 times, or display all numbers between 1 and 100. For such situations, Python provides a built-in function called `range()`.

What Is range()?

The `range()` function generates a sequence of integers (whole numbers). It does not store all the numbers in memory — it produces them one at a time as the loop needs them. This makes it very fast and memory-friendly, even when you need a very large range of numbers.

Three Forms of range()

Form	Meaning	Example
<code>range(stop)</code>	Numbers from 0 up to (but not including) stop.	<code>range(5)</code> → 0,1,2,3,4
<code>range(start, stop)</code>	Numbers from start up to (but not including) stop.	<code>range(2, 6)</code> → 2,3,4,5
<code>range(start, stop, step)</code>	Numbers from start to stop, jumping by step.	<code>range(1, 10, 2)</code> → 1,3,5,7,9

Remember

The stop value is never included in the output. So `range(5)` gives five numbers: 0, 1, 2, 3, 4 — it stops just before reaching 5.

Example 1: Using range(stop)

```
for i in range(5):  
    print("Iteration number:", i)
```

Output:

```
Iteration number: 0  
Iteration number: 1  
Iteration number: 2  
Iteration number: 3  
Iteration number: 4
```

Example 2: Using range(start, stop)


```
for i in range(1, 6):  
    print("Square of", i, "is", i * i)
```

Output:

```
Square of 1 is 1  
Square of 2 is 4  
Square of 3 is 9  
Square of 4 is 16  
Square of 5 is 25
```

Example 3: Using range(start, stop, step)

The third value, called step, tells the loop how much to increase each time. By default it is 1, but we can change it.

```
# Print all even numbers between 2 and 20  
for i in range(2, 21, 2):  
    print(i, end=" ")
```

Output:

```
2 4 6 8 10 12 14 16 18 20
```

Example 4: Counting Backwards

If the step is a negative number, the loop will count downwards. This is useful for making countdowns.

```
for i in range(10, 0, -1):  
    print(i)  
  
print("Blast off!")
```

Output:

```
10  
9  
8  
7  
6  
5  
4  
3  
2  
1  
Blast off!
```


5. Iterating Over Strings

In Python, a string is a sequence of characters — just like a word is a sequence of letters. Because strings are sequences, we can use a for loop to go through each character one at a time.

Example 1: Printing Each Character

```
word = "Python"

for letter in word:
    print(letter)
```

Output:

```
P
y
t
h
o
n
```

The loop picks one character at a time from the string "Python" and stores it in the variable letter. Spaces, punctuation and digits are also treated as characters.

Example 2: Counting the Vowels in a Word

```
name = "Education"
count = 0

for ch in name:
    if ch in "aeiouAEIOU":
        count = count + 1

print("Number of vowels:", count)
```

Output:

```
Number of vowels: 5
```

Here the variable ch takes each character from the word. The condition checks whether that character is a vowel. If yes, count is increased by 1.

Example 3: Reversing a String

```
text = "HELLO"
reversed_text = ""
```

```
for ch in text:  
    reversed_text = ch + reversed_text  
  
print("Reversed string:", reversed_text)
```

Output:

Reversed string: OLLEH

In this example, each new character is placed before the existing text. By the end of the loop, the string is fully reversed. This is a small but very interesting use of the for loop.

**Good to Know**

Strings are immutable in Python, which means you cannot change the original string while looping. You can only build a new string, as we did above.



6. Iterating Over Lists, Tuples and Sets

Lists, tuples and sets are three very common collections in Python. A for loop works with all of them in almost the same way. Let us explore each one with examples.

6.1 Iterating Over a List

A list is a collection of items enclosed in square brackets []. The items are kept in a fixed order, and the list can be changed later (for example, you can add or remove items).

```
students = ["Ravi", "Priya", "Neha", "Arjun"]

for student in students:
    print("Welcome,", student)
```

Output:

```
Welcome, Ravi
Welcome, Priya
Welcome, Neha
Welcome, Arjun
```

The loop goes through the list in the exact order in which the items are stored.

Using enumerate() to Get Both Index and Value

Sometimes you also need to know the position (index) of each item. The built-in function enumerate() makes this simple.

```
students = ["Ravi", "Priya", "Neha", "Arjun"]

for index, student in enumerate(students):
    print(index, "-", student)
```

Output:

```
0 - Ravi
1 - Priya
2 - Neha
3 - Arjun
```

6.2 Iterating Over a Tuple

A tuple is similar to a list, but it is written with parentheses () and cannot be changed after it is created. Tuples are commonly used for fixed collections — like the days of the week or coordinates on a map.

```
days = ("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun")
```

```
for day in days:  
    print(day)
```

Output:

```
Mon  
Tue  
Wed  
Thu  
Fri  
Sat  
Sun
```

From the programmer's point of view, iterating over a tuple feels exactly like iterating over a list.

6.3 Iterating Over a Set

A set is a collection of unique items written inside curly braces { }. Sets do not keep items in any specific order and do not allow duplicates.

```
colors = {"red", "green", "blue", "yellow"}
```

```
for color in colors:  
    print(color)
```

Output:

```
red  
blue  
yellow  
green
```



Please Note

Because a set has no fixed order, the output may appear in a different sequence each time you run the program. That is completely normal.

Quick Comparison

Collection	Symbol Used	Special Property
List	[]	Ordered and changeable.
Tuple	()	Ordered but cannot be changed.
Set	{ }	Unordered and does not allow duplicates.

7. Iterating Over Dictionaries

A dictionary in Python is a collection of key-value pairs. It is written with curly braces { } where each item has two parts separated by a colon — the key and the value. Dictionaries are very useful when you want to connect one piece of information with another, like a student's name and their marks.

A Simple Dictionary

```
marks = {  
    "Ravi": 85,  
    "Priya": 92,  
    "Neha": 78,  
    "Arjun": 88  
}
```

7.1 Iterating Over the Keys

By default, a for loop on a dictionary goes through its keys.

```
for name in marks:  
    print(name)
```

Output:

```
Ravi  
Priya  
Neha  
Arjun
```

We can also use `.keys()` to make it more readable, although it is optional.

```
for name in marks.keys():  
    print(name)
```

7.2 Iterating Over the Values

If we are interested only in the values, we use `.values()`.

```
for score in marks.values():  
    print(score)
```

Output:

```
85  
92  
78
```

7.3 Iterating Over Both Keys and Values

To access both at the same time, we use `.items()`. It returns each entry as a pair.

```
for name, score in marks.items():  
    print(name, "scored", score, "marks")
```

Output:

```
Ravi scored 85 marks  
Priya scored 92 marks  
Neha scored 78 marks  
Arjun scored 88 marks
```

A Practical Example

Let us find which students have scored more than 80 marks.

```
marks = {"Ravi": 85, "Priya": 92, "Neha": 78, "Arjun": 88}  
  
print("Students with distinction (above 80):")  
for name, score in marks.items():  
    if score > 80:  
        print(name, "-", score)
```

Output:

```
Students with distinction (above 80):  
Ravi - 85  
Priya - 92  
Arjun - 88
```

**Tip**

If you only need the keys, iterate directly over the dictionary. Use `.items()` only when you also need the values. This makes your code clearer and slightly faster.

8. Nested for Loops

A loop placed inside another loop is called a nested loop. This is useful when we need to work with two levels of data — for example, rows and columns of a table, or pairs of items from two different collections.

How Nested Loops Work

For every single turn of the outer loop, the inner loop runs completely from start to end. If the outer loop runs 3 times and the inner loop runs 4 times, the inner body will be executed $3 \times 4 = 12$ times in total.

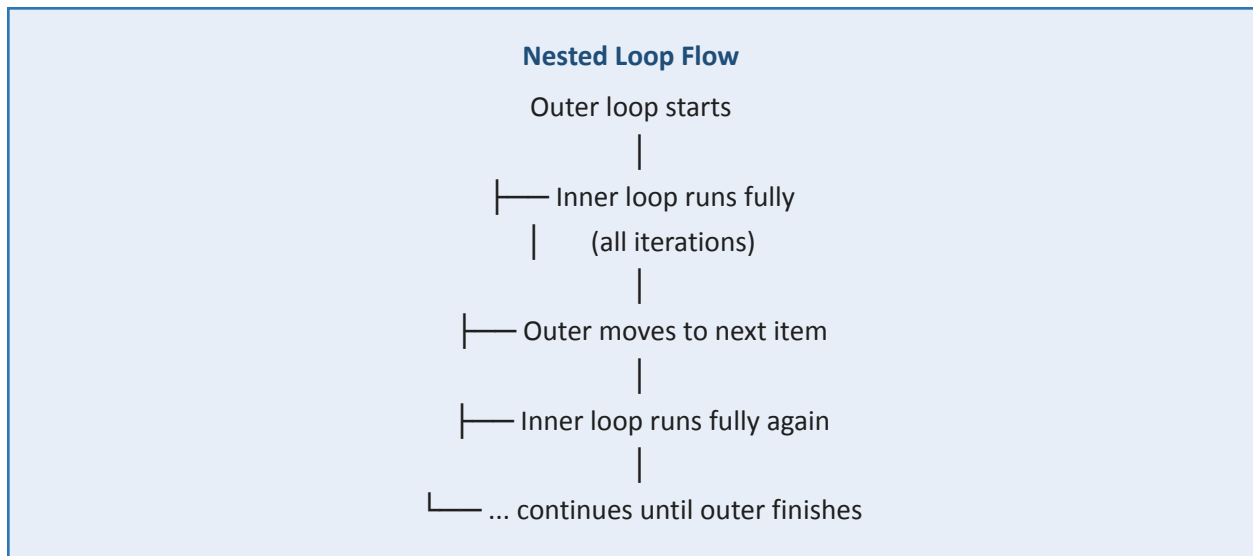


Fig. 8.1 — Working of nested for loops.

Example 1: Printing a Simple Pattern

```
for i in range(1, 4):
    for j in range(1, 4):
        print("*", end=" ")
    print() # moves to the next line
```

Output:

```
* * *
* * *
* * *
```

Here the outer loop controls how many rows to print, and the inner loop controls how many stars in each row.

Example 2: Multiplication Table

```
for i in range(1, 4):  
    print("Table of", i)  
    for j in range(1, 6):  
        print(i, "x", j, "=", i * j)  
    print()
```

Output:

Table of 1
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5

Table of 2
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10

Table of 3
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15

Example 3: A Number Triangle

```
for i in range(1, 6):  
    for j in range(1, i + 1):  
        print(j, end=" ")  
    print()
```

Output:

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

In this program, the number of items printed by the inner loop depends on the current value of the outer loop. This is a very common technique for drawing patterns.

 Caution

Nested loops increase the total work very quickly. A single outer loop with N items and an inner loop with M items will run $N \times M$ times. Be careful when working with very large collections.



9. Using break with the for Loop

Sometimes we want to stop a loop in the middle — before it has finished going through all the items. The `break` statement allows us to do exactly that. As soon as Python sees `break`, it exits the loop immediately and moves on to the next line of code after the loop.

When to Use `break`

- When we have found what we were searching for and no longer need to continue.
- When a certain condition tells us that further looping is unnecessary.
- To save time, especially when the sequence is large.

Example 1: Searching for an Item

```
students = ["Ravi", "Priya", "Neha", "Arjun", "Kiran"]
search = "Neha"

for student in students:
    if student == search:
        print(search, "is in the class.")
        break
    print("Checked:", student)
```

Output:

```
Checked: Ravi
Checked: Priya
Neha is in the class.
```

Once Neha is found, the `break` statement stops the loop. The remaining names, Arjun and Kiran, are not checked at all.

Example 2: Stopping at the First Negative Number

```
numbers = [4, 8, 15, -2, 10, 20]

for num in numbers:
    if num < 0:
        print("Negative number found:", num)
        break
    print("Processing:", num)
```

Output:

```
Processing: 4
Processing: 8
```

Processing: 15
Negative number found: -2

 Tip

If a for loop is nested inside another for loop, the break statement only exits the inner loop — not the outer one.



10. Using continue with the for Loop

The continue statement is a little different from break. Instead of stopping the loop entirely, it skips only the current iteration and moves on to the next one. The rest of the items are still processed normally.

When to Use continue

- When we want to ignore certain items but still continue with the others.
- When some data is invalid and must be skipped.
- To make our logic cleaner by avoiding deeply nested if statements.

Example 1: Skipping Even Numbers

```
for num in range(1, 11):  
    if num % 2 == 0:  
        continue  
    print(num)
```

Output:

```
1  
3  
5  
7  
9
```

Whenever the number is even, continue sends control back to the top of the loop. The print() statement is never reached for even numbers.

Example 2: Ignoring a Specific Value

```
marks = [78, 85, -1, 92, -1, 88, 70]  
total = 0  
valid_count = 0  
  
for m in marks:  
    if m == -1:    # -1 means the student was absent  
        continue  
    total = total + m  
    valid_count = valid_count + 1  
  
average = total / valid_count  
print("Average marks:", average)
```

Output:

```
Average marks: 82.6
```

In this example, the value -1 is used to represent an absent student. The continue statement skips such values, so they do not affect the average.

break vs continue — A Quick Comparison

break	continue
Completely stops the loop.	Skips only the current iteration.
Control moves out of the loop.	Control goes back to the loop header.
Remaining items are not processed.	Remaining items are still processed.
Used when we have found a result.	Used when we want to ignore some items.



11. Using pass with the for Loop

The pass statement is a special keyword that does absolutely nothing. At first it may feel strange to have a statement that does nothing — but it is surprisingly useful while developing and planning programs.

Why Do We Need pass?

In Python, a for loop must always have at least one statement inside it. If you leave the loop body empty, Python will give an error. The pass statement acts as a placeholder — it fills the empty spot so that the code runs without any error, even when you have not yet decided what to put there.

Example 1: An Empty Loop Body

```
for i in range(5):  
    pass # To be filled in later
```

This loop runs 5 times but does nothing each time. It is perfectly valid Python code.

Example 2: Using pass While Planning Code

```
numbers = [1, 2, 3, 4, 5]  
  
for n in numbers:  
    if n % 2 == 0:  
        pass # I will write the even-number logic later  
    else:  
        print(n, "is odd")
```

Output:

```
1 is odd  
3 is odd  
5 is odd
```

Here, we have not yet decided what to do with even numbers, so we put pass to avoid errors. The code still runs, and we can come back later and complete the logic.

break, continue and pass — Side by Side

Statement	Effect on the Loop	Typical Use
break	Stops the loop right away.	Exit when a goal is reached.
continue	Skips the current iteration only.	Ignore certain items and move on.

Statement	Effect on the Loop	Typical Use
pass	Does nothing — acts as a placeholder.	Use when you need empty code temporarily.

**Remember**

pass is different from a comment. A comment (# like this) is completely removed when the program runs, while pass is an actual Python statement that the interpreter executes — it just has no effect.



12. else with the for Loop

Python has a feature that many beginners find surprising — a for loop can have an else block attached to it. This is not available in many other programming languages, and it can be very powerful when used correctly.

How It Works

The else block after a for loop runs only when the loop completes normally — that is, when it finishes going through all the items without being stopped by a break statement. If the loop is interrupted by break, the else block will not run.

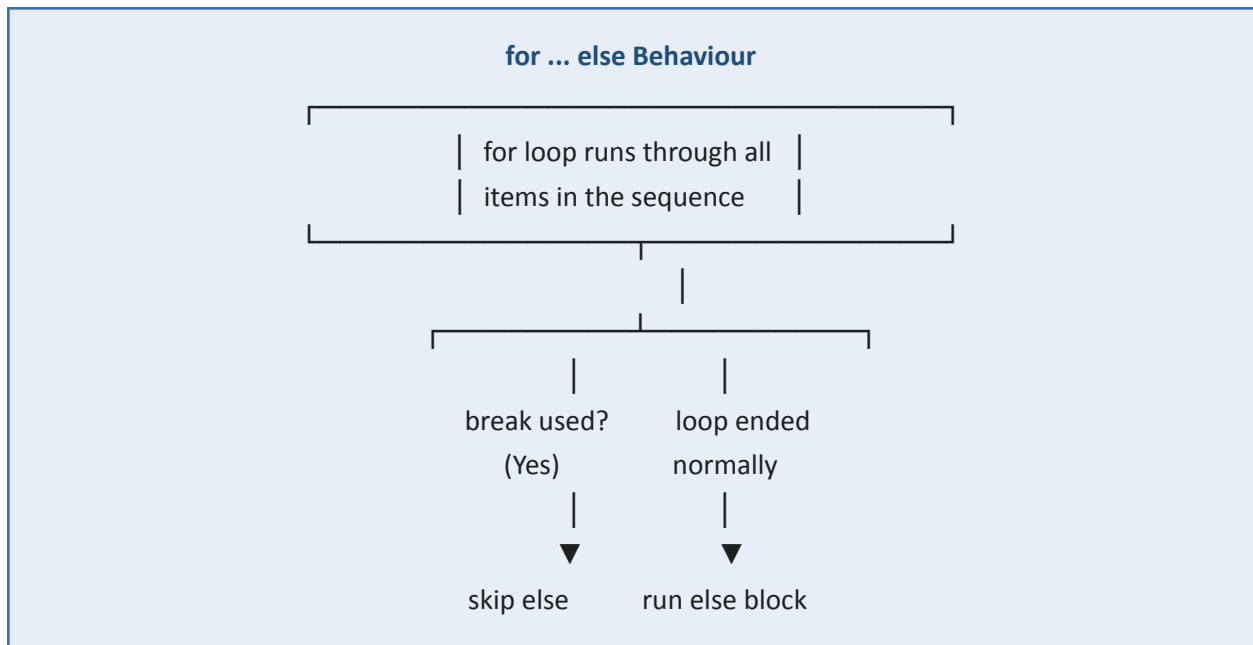


Fig. 12.1 — When the else block runs after a for loop.

Example 1: Loop Completes Normally

```
for i in range(1, 4):
    print("i =", i)
else:
    print("Loop finished without break.")
```

Output:

```
i = 1
i = 2
i = 3
Loop finished without break.
```

Since the loop was never interrupted, the else block ran after the final iteration.

Example 2: Loop Stopped by break

```
for i in range(1, 6):
    if i == 3:
        print("Breaking out!")
        break
    print("i =", i)
else:
    print("This line will NOT be printed.")
```

Output:

```
i = 1
i = 2
Breaking out!
```

Here the loop was stopped by break, so the else block did not run at all.

A Practical Example: Searching in a List

```
numbers = [12, 45, 78, 34, 90]
target = 50

for n in numbers:
    if n == target:
        print(target, "found in the list.")
        break
else:
    print(target, "was not found in the list.")
```

Output:

```
50 was not found in the list.
```

This is a neat way of writing search programs. If the target is found, break exits the loop and skips else. If the target is not found, the loop ends normally and the else block announces that the item was missing.



Important

The else block in a for loop is not an error-handling block. It simply means "run this only if the loop was not broken". Do not confuse it with else used after an if statement.

13. Chapter Summary

In this chapter we have studied the for loop of Python — one of the most important tools every programmer uses. Here is a quick recap of the key points we covered:

- A for loop is used to repeat a block of code for each item in a sequence.
- The correct syntax is for variable in sequence: followed by an indented block.
- A for loop can iterate over strings, lists, tuples, sets and dictionaries.
- The range() function creates a sequence of numbers and is very useful for counting.
- A loop placed inside another loop is called a nested loop.
- break stops the loop immediately, continue skips the current iteration, and pass does nothing and acts as a placeholder.
- The optional else block runs only when the loop ends without using break.

In the Next Chapter

We will look at Python's while loop — another way of repeating code, especially useful when you do not know in advance how many times the loop should run. We will also compare for and while loops so that you know when to use which.

